# [Experiment, Analysis, and Benchmark] Filtered Vector Search Under Updates: When Does an Index Beat a Scan?

Suan Lee
Semyung University, Jecheon, Republic of Korea
suanlee@semyung.ac.kr ORCID: https://orcid.org/0000-0002-3047-1167

***Abstract*—Vector databases answer nearest-neighbor queries with attribute filters over data that is continuously inserted and deleted, yet filtered search has been evaluated mostly on static data. We evaluate eleven methods from nine families on SIFT, DEEP, YFCC, and 4096-dimensional arXiv collections of up to $10^7$ vectors, with controlled and real predicates, on three synthetic update streams and one with real temporal drift, and inside PostgreSQL and Milvus. Two lessons emerge. First, the cheapest access path depends on conditions that benchmarks usually fix. The number of points passing the filter at which an index overtakes an exact scan grows with the collection (on DEEP, $n^{0.41}$ to $n^{0.64}$ for IVF, ACORN, and post-filtering). A dispatch threshold fixed as a size or a selectivity, such as Milvus's 7% applied to our choice between scan and IVF, therefore picks the slower path at some sizes, while one scaled with $\sqrt{n}$ transferred across sizes and, corrected by a dataset-free microbenchmark, to a second machine. Over a stream, the cheapest method depends on the query-to-update ratio and the data: a label-partitioned IVF (one IVF index per label) on our synthetic streams, graph indexes on the drifting arXiv stream from about one query per update. Second, deletions break filtered search in different ways. ACORN, which only masks deleted points, loses recall when deletions correlate with the attribute (1.000→0.575), as the share of a label reachable from its search's entry point shrinks; repairing its neighbor lists on every level restores it by the last round (0.991–0.998, 8 runs) but lowers recall under uniform deletions on DEEP-5M. EMA's synchronous deletion keeps recall, but its throughput drops 5.3× under correlated deletions; IVF's exact deletions keep both at two to three orders of magnitude less maintenance. A cost model frames the crossover through a loose $\sqrt{n}$ lower bound for indexes that check inadmissible points. Pre-registered follow-ups test both lessons, and every number is regenerated from the released artifact.**

***Index Terms*—vector search, approximate nearest neighbor search, filtered search, hybrid queries, dynamic indexing, benchmark**

## I. Introduction

Vector databases answer nearest-neighbor queries that carry attribute predicates—"similar products *in stock*", "passages *from this tenant*"—over collections that change continuously. A large body of recent work builds indexes for one of these two requirements: filter-aware graphs that keep navigation inside a predicate [1], [2], [3] or fold attribute mismatch into the graph distance [4], range-filter structures [5], [6], [7], and streaming indexes that absorb insertions and deletions [8], [9], [10]. Experimental studies have compared filtered methods on static data [11], [12], [13], [15], [16], and benchmarks have exercised streaming insertions and deletions [17] and continuous insertion [18], without filters. A deployment, however, faces both at once, and the experimental study of Li et al. lists dynamic updates as an open problem for filtered search [12]. This paper evaluates filtered search *under updates*, against simple baselines—an exact scan, a bitmap-filtered IVF index, and a selectivity dispatcher—that are easy to deploy alongside any index.

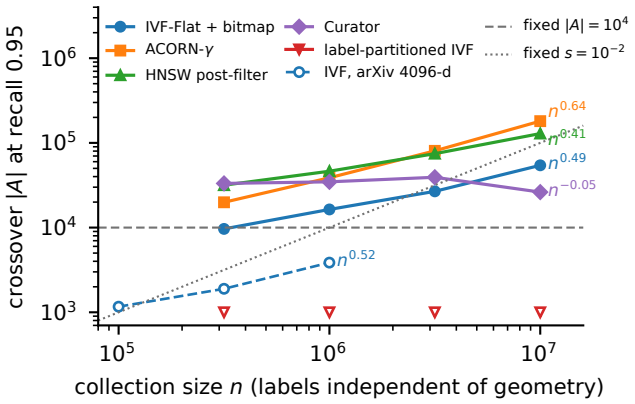



Fig. 1. Where an index catches up with an exact per-label scan, as the collection grows (recall 0.95, labels independent of geometry; DEEP subsets, and IVF on 4096-dimensional arXiv embeddings). Above its curve an index beats the scan. A dispatch threshold stated as a fixed admissible-set size (dashed) or a fixed selectivity (dotted) crosses the curves, so it does not track the measured crossover. Exponents of log–log fits; hollow markers: the index already wins at the smallest measured size ($10^3$).

**The crossover moves with the collection.** Systems place the switch between a scan and an index with a fixed rule: pre-filter below a selectivity (Milvus 2.6: 7% [19], [20]; ACORN [1]), or scan rare tags linearly (the winning Big-ANN'23 filtered entry [17]). Mandarapu and Kunkunuru observed for pre- versus post-filtering under a fixed budget that the crossover selectivity falls as $1/\sqrt{n}$ [21]. We find that a growing crossover holds across index families and decides which rule to deploy (Figure 1). Under independent admissibility (Section IV), an index that must check inadmissible points pays for each one it examines, so the admissible-set size at which it catches up with an exact scan has a lower bound that grows as $\sqrt{n}$, and the measured one grows (lesson (1)

below). Choosing between scan and IVF on the other DEEP sizes, a fixed admissible-set threshold calibrated on DEEP-1M picks the slower path on 2 of 17 workloads, Milvus's 7% rule on 5, and the calibration scaled with $\sqrt{n}$ on none; where the check-to-distance cost ratio differs (arXiv, a second machine), the scaled threshold also needs that ratio, which a dataset-free microbenchmark supplies (exploratory on arXiv, pre-registered on the second machine). Updates add a second mechanism—under correlated deletions an in-filter graph loses the routes its search takes into a label's region—and the rest of the paper measures both.

We ask six questions. **Q1**: When does an index beat an exact scan of the admissible points, and is the crossover a function of the admissible-set size $s \cdot n$? **Q2**: How does the correlation between attributes and geometry reorder the methods? **Q3**: What happens on real predicates whose selectivity spans orders of magnitude? **Q4**: How do accuracy, throughput, footprint, and maintenance evolve as the data churns, and what if deletions correlate with the attribute? **Q5**: Which method is cheapest once maintenance and queries are both paid for? **Q6**: Which evaluation practices change the conclusions?

**Two lessons.** Both show that the winning method depends on parameters that benchmarks hold fixed: the collection size, two correlations, the query-to-update ratio, and the deletion pattern. *(A) The cheapest access path is conditional.* (1, **Q1**) The crossover grows with $n$ for IVF, ACORN, and post-filtering, not for label-indexed structures (Figure 1; IVF's exponent is 0.41–0.48 over 7 label and training seeds, and recurs on arXiv). (2, **Q2**) Correlation reorders the methods—post-filtering fails to reach recall $0.95$ with correlated attributes, a label-specialized graph succeeds—but only when queries ask for labels whose region lies far from them. (3, **Q3**) On YFCC-10M's tags no index reaches recall $0.99$ alone; a tuned selectivity dispatcher gets every index there, and on 500 held-out queries IVF, post-filtering, and RIG still hold it while ACORN, Curator, and Filtered-DiskANN fall to 0.984–0.989. (4, **Q5**) Over a stream, a label-partitioned IVF is cheapest at every tested query-to-update ratio on the synthetic streams (0.22–0.39× IVF), and graph indexes are cheapest on the drifting arXiv stream from about one query per update. *(B) Deletions break filtered search in different ways* (Section VI). (5, **Q4**) ACORN, which deletes only through its predicate, collapses (maximum recall 1.000→0.575) when deletions correlate with the attribute; a repair of every level restores it (0.991–0.998 at the last round, 8 runs, three stream seeds) but lowers recall under uniform deletions on DEEP-5M when it re-ranks a node's live neighbors, while refilling only the deleted slots does not. Across these update paths a label's recall follows the share of its live nodes reachable from the entry (Figure 3). EMA's synchronous deletion keeps recall but loses 5.3× throughput under correlated deletions; deleting by age on arXiv breaks no graph. *Robustness.* Pre-registered follow-ups (Table V) re-test the decisive comparisons across seeds, randomized blocks, timing boundaries, concurrency, further streams, a second machine, every-round traces, paired blocks, and 132 re-executed jobs. They support the dispatch rule, the repair's trade-off, and the arXiv ranking of graphs over IVF; they reject a $\sqrt{n}$ exponent for IVF and post-filtering, one mechanism account, the repair's recall at every round, two close verdicts, and EMA's lead under a fixed search effort. Evaluation practices that changed conclusions are in Section VIII.

TABLE I
THE CLOSEST PRIOR EVALUATIONS, BY WHAT THEIR EXPERIMENTS COVER (✓ YES, ○ PARTLY, – NO)

| study | filters | inserts and deletes interleaved, recall tracked | deletions correlated with the attribute |
|---|---|---|---|
| Zhu [11], Li [12], Shi [13], Iff [15], Lu [16], Mandarapu [21] | ✓ | – | – |
| VecBench [47] | ✓ | ○ latency, not recall | – |
| DSG [45], WoW [46], DIGRA [43], Zhang [44] | ✓ | ○ insert- or delete-only | – |
| EMA [25], Curator [23] | ✓ | ○ own method only | – |
| Big-ANN'23 streaming [17] | – | ✓ | – |
| CANDY [18] | – | ○ inserts only | – |
| this paper | ✓ | ✓ | ✓ |

**Contributions.** (i) A benchmark that records each method's predicate and update semantics and measures recall and cost while insertions and deletions interleave, including deletions correlated with the attribute (Section III; closest studies in Table I). (ii) A cost model whose crossover bound, reachability account, and rebuild rule were tested against criteria stated in advance (Section IV, Table V). (iii) A study of eleven methods and two DBMSs on static workloads and of nine methods on four update streams, one with real drift (Sections V–VII). (iv) Evaluation lessons, a fix for an ACORN correctness bug, an AVX2 build replacing its scalar default, a repair for its deletions, and an artifact that regenerates every number (Section VIII).

## II. FILTERED SEARCH AND ITS UPDATE PATHS

A filtered $k$-NN query asks for the $k$ points nearest to $q$ among the *admissible set* $A_p = \{x \in X : p(x)\}$ of a predicate $p$; its selectivity is $s = |A_p|/n$. We group the methods we evaluate into nine families by *where the predicate enters the search* and record, for each, *how it absorbs insertions and deletions* (Table II).

**Pre-filter scan.** Compute $A_p$, then scan it exactly. When the predicate is an equality on a label known at insertion time, the points of each label can be stored contiguously (*per-label partition*), so the scan touches only $|A_p|$ vectors; we also evaluate the general case in which $A_p$ is an arbitrary bitmap and the scan visits all $n$ rows and skips inadmissible ones (*bitmap scan*). Updates are appends and removals; there is nothing to repair.

**Post-filter.** Search an unfiltered index for $k' \geq k$ candidates and keep the admissible ones. We use hnswlib [22] with slot reuse for deletions. Post-filtering needs $k' \gtrsim k/s$ when admissible points are spread like the data, and fails when they are not.

**Predicate-agnostic in-filter graph.** ACORN-$\gamma$ [1] builds an HNSW-like graph whose neighbor lists are expanded by a factor $\gamma$, so that the subgraph induced by an arbitrary predicate stays navigable; search visits only admissible nodes. Insertions use the graph's incremental `add`; ACORN has no deletion routine, so deleted points are excluded through the predicate bitmap and remain in the graph as routers. We also evaluate ACORN-1 ($\gamma$=1), its cheaper-to-build variant.

**Label-specialized graph.** Filtered-DiskANN [2] builds a Vamana graph whose pruning preserves, for every label, paths among the points carrying it, and starts each query from a per-label start point. It supports equality on build-time labels only. The public Python binding exposes no dynamic filtered index, so freshness requires rebuilding.

**Label-indexed tree.** Curator [23] builds one hierarchical clustering tree over all vectors and embeds a lightweight per-label index in it as a subtree; a point is indexed under a label by granting it the label, and a label search traverses that label's subtree. Other predicates, such as the AND of two tags, use its bitmap-filtered search. Its label ids are 16-bit, so on YFCC we index the tags that occur in the queries (a build informed by the workload). Insertion adds the vector to the tree and grants its labels. The released implementation rejects vector removal, so deletion revokes the point's labels: the vector stays in the tree, but no label search can return it.

**Partition with bitmap.** IVF-Flat [24] assigns points to the nearest of $n_{\text{list}}$ k-means centroids and scans the $n_{\text{probe}}$ lists nearest to the query, skipping inadmissible entries via a bitmap. With centroids fixed after training, insertion is an append to one list and deletion an exact removal; there are no tombstones and no graph to repair. Correlated predicates hurt IVF in a specific way: the admissible points may live in lists far from the query, so recall at a fixed $n_{\text{probe}}$ drops.

**Label-partitioned IVF.** A design point that the cost model of Section IV suggests and that we add to the study: one IVF-Flat index per label over that label's points ($n_{\text{list}} = 4\sqrt{n_b}$ for a label with $n_b$ points, each with its own k-means). A label search probes only that label's lists, so it never examines an inadmissible point, and insertion and deletion are exact, as in IVF. It requires an equality predicate on a build-time label.

**Marker graph (EMA).** EMA [25] attaches to each HNSW edge a small signature of the attributes reachable through it, so that filtered search skips edges that cannot lead to admissible points, and deletes by an IP-DiskANN-style in-place edge repair [10], extended to the signatures (the preprint describes lazy batched deletion; the public code we run repairs inside each deletion call). We run the authors' code (commit d95fb49) with the parameters of their dynamic experiments ($M$=40, $ef_c$=300).

**Regenerable pool index.** RIG, our own unpublished design point, stores no adjacency lists: points belong to overlapping pools, neighbors are recomputed at query time inside the predicate, and deletions are exact.

TABLE II
METHOD FAMILIES, HOW EACH CONSUMES THE PREDICATE, AND HOW EACH ABSORBS UPDATES IN THE IMPLEMENTATIONS WE EVALUATE.

| family | method (impl.) | predicate | insert | delete |
|---|---|---|---|---|
| pre-filter scan | per-label / bitmap (Faiss) | any[†] | append | remove |
| post-filter | HNSW (hnswlib) | any | add | slot reuse |
| in-filter graph | ACORN-$\gamma$, ACORN-1 | any | add | bitmap exclusion |
| marker graph | EMA (authors' code) | DNF[§] | add | repair |
| label graph | Filtered-DiskANN (diskannpy) | eq. label | rebuild | rebuild |
| label tree | Curator | label[‡] | add+grant | revoke label |
| partition+bitmap | IVF-Flat (Faiss) | any | append | remove |
| label partition | label-partitioned IVF (Faiss) | eq. label | append | remove |
| regenerable pools | RIG (ours) | any | route+append | remove |

[†]The per-label partition requires an equality predicate on a build-time label; the bitmap scan accepts any. [‡]Other predicates through a bitmap-filtered search. [§]Numeric ranges and label sets; evaluated on the update streams only.

## III. BENCHMARK DESIGN

**Datasets and predicates.** We use SIFT1M [26] ($n$=$10^6$, $d$=128) and DEEP [27] (both as distributed by ANN-Benchmarks [28]) at $10^6$, $3.16 \times 10^6$, and $10^7$ points ($d$=96, normalized; the two smaller sets are fixed random subsets of DEEP-10M, so that $n$ varies without changing the distribution), and YFCC-10M from the filtered track of the NeurIPS'23 Big-ANN competition [17], derived from YFCC100M [29] ($n$=$10^7$, $d$=192), whose points carry real tag sets and whose queries ask for the AND of one or two tags. On SIFT and DEEP we generate label predicates with two controlled knobs. Each point receives one of $L = 1/s$ labels, defined by quantiles of $\beta\, z(x) + (1-\beta)\,\varepsilon$, where $z(x)$ is the standardized projection $\langle x, w\rangle$ on a fixed random direction $w$ and $\varepsilon$ is standard Gaussian noise, so that $s$ sets the selectivity and $\beta \in [0,1]$ controls the correlation between label and geometry ($\beta$=0: independent labels; $\beta$=1: slabs along $w$); each query asks for one uniformly random label. We use $s \in \{0.5, 0.1, 0.01\}$ and $\beta \in \{0, 0.5, 0.9, 1\}$. On YFCC we use 500 fixed random public queries; their selectivities range from $7.4\times10^{-6}$ (74 admissible points) to 0.19, with 10th, 50th, and 90th percentiles of $2.2 \times 10^{-5}$, $1.3 \times 10^{-3}$, and $8.1 \times 10^{-2}$, and our exact filtered ground truth matches the official one on all of them.

**Update streams.** Three synthetic streams combine filtered queries ($\beta$=0.9, $s$=0.1) with churn, and a fourth orders real data by time. *SIFT-500K*: SIFT descriptors drawn from BIGANN [30] ($1.5\times10^6$ distinct vectors, 500 BIGANN queries), $5 \times 10^5$ live points, 20 rounds, each deleting 10% of the live points uniformly at random and inserting $5\times10^4$ unseen points (200% turnover). *DEEP-5M*: $5 \times 10^6$ live points, 10 rounds of 10% deletions and $4.99 \times 10^5$ insertions of unseen DEEP vectors (100% turnover). A third stream, *SIFT-500K-adv*, tests correlation $\times$ churn: it is SIFT-500K with the same insertions and queries, but each round deletes the 10% of live points with the largest projection on the label direction $w$ (plus small noise), so deletions empty the geometric regions of the highest labels first and the admissible sets of those labels shrink while others grow. *arXiv-time*: the arXiv embeddings ordered by the date of each paper's latest version; the $2\times10^5$ oldest are live at

round 0, and each of 10 rounds inserts the next $4 \times 10^4$ papers and deletes the $4 \times 10^4$ oldest live ones. A paper's label is its first main category, mapped to the 9 most frequent categories of the initial window plus 'other' (so no later information enters the mapping), and the 500 queries are later papers asking for their own label, fixed across rounds—a stress test of drift, not a replay of arXiv's traffic. Two further streams appear only in Table V: *Amazon-time*, 384-dimensional all-MiniLM-L6-v2 embeddings of reviews from ten categories of Amazon Reviews 2023 [14], in review order with arXiv-time's window and schedule and the category as label; and a multi-label YFCC stream, $10^6$ YFCC-10M points with their tag sets, 10 rounds of $10\%$ uniform deletions and as many unseen insertions, queried with the AND predicates of 500 public queries. Exact filtered ground truth is recomputed over the live set after every round. As a control for the update paths, we also snapshot the live set of both SIFT streams after round 20 and build every method from scratch on it, with the same queries and ground truth. Every method applies every round through its own update path (Table II), except the snapshot-rebuilt Filtered-DiskANN, which is rebuilt at the measured rounds only; search quality is measured at rounds $0, 4, \ldots, 20$ (SIFT) and $0, 5, 10$ (DEEP, arXiv).

**Follow-up campaign.** A second campaign under the same protocol tests the cost model; later sessions (repair variants, the label-partitioned IVF and EMA on the streams, the executed rebuild policies, the arXiv stream) each rerun IVF alongside and are combined with others only as ratios to that session's IVF. It runs in two passes unless a range or run count is given. It adds (i) an $n$-sweep with labels independent of geometry ($\beta$=0): DEEP subsets of $3.16 \times 10^5$, $10^6$, $3.16 \times 10^6$, and $10^7$ points with $|A_p| = n/L$ from about $10^3$ to $3.16 \times 10^5$; (ii) query–filter correlation on SIFT1M (with probability $\rho \in \{0, 0.5, 1\}$ a query asks for its nearest neighbor's label); (iii) counts of checks and distances (IVF per query, ACORN in a separate build); (iv) instrumented ACORN streams that record each label's live admissible subgraph, and a local repair of it; (v) dense sweeps (knob steps of $\sqrt{2}$) where conclusions depend on interpolation (Section VIII); (vi) PostgreSQL 16 with pgvector 0.8.1 (Section V-E; HNSW with $m$=32, $ef_c$=200, and IVFFlat with $4\sqrt{n}$ lists, both with iterative scans); and (vii) 4096-dimensional arXiv embeddings [15] ($10^5$, $3.16 \times 10^5$, and $10^6$ points, 500 of the dataset's queries) with $\beta$=0 labels and, at $10^6$, the real predicate "category includes the query's".

**Parameters.** Each method is swept over its search knob until recall exceeds $0.999$ or a cap is reached: ACORN `efSearch` 16..2048, Filtered-DiskANN $L$ 16..1024, IVF $n_{\text{probe}}$ $8..\min(2048, n_{\text{list}}/2)$ with $n_{\text{list}} = 4\sqrt{n}$, post-filter $k' \in \{100, 400, 1600, 6400\}$ (streams: $\{400, 1600, 6400\}$), RIG $ef$ 16..512; the same ranges apply to the streams. Sweeps and dispatcher thresholds are tuned on the reported queries, as is customary (held-out checks: Sections V-C, IX). Construction parameters follow the original papers' defaults: ACORN $M$=32, $\gamma$=12, $M_\beta$=64; Filtered-DiskANN $R$=64, $L$=100, $\alpha$=1.2; HNSW $M$=32, ef$_c$=200.

**Metrics.** Every query asks for $k$=10 neighbors. Recall@10 of a query is the fraction of its exact filtered top-10 that is returned (over $\min(10, |A_p|)$ when fewer points are admissible), averaged over 500 queries; *maximum recall* is the highest recall in a method's parameter sweep, and QPS at a recall target is interpolated log-linearly on the method's recall–QPS Pareto frontier, except for the dispatcher of Section V-C, whose QPS is that of the best measured configuration (a concrete threshold and parameter) that reaches the target. QPS is the number of queries divided by the time spent inside the search calls of a single thread.

**Measurement protocol.** Searches are single-threaded and issue one query per call, with these exceptions: on label workloads ACORN and EMA are called once for all 500 queries (on one thread) and RIG once per label, and post-filtering, the bitmap scan, and the per-label scan are called once per label (the per-label scan is also reported per query); one call per label lets BLAS share work across queries. Overheads at the timer boundary are small: post-filtering's candidate selection (56 $\mu$s at $k'$=6400), Filtered-DiskANN's second-tag check (73 $\mu$s), and the call into RIG's kernel (354 $\mu$s, favoring RIG) are outside it, ACORN's visited-table reset on YFCC (0.40 ms per call, disfavoring it) is inside, and binding calls are a median 2% of query time (at most 4% at recall $\geq 0.95$). Every method receives its predicate precomputed (label, bitmap, or admissible list), so predicate evaluation is not timed (its cost: Section IX). Static builds use 32 threads; in the streams, initial builds, update maintenance, and rebuilds use 16 threads, uniformly.

Each configuration is warmed once and timed three times; the median 95% interval is $\pm$0.8% of the mean with a normal approximation and $\pm$2.0% with Student's $t$ for three timings (90th percentiles $\pm$3.8% and $\pm$10%). The machine, a two-socket Xeon Gold 6258R (2×28 cores, 38.5 MB LLC per socket, 502 GB RAM; Ubuntu 22.04, kernel 6.8, GCC 11.4), was shared with unrelated workloads, so within-job intervals do not capture the dominant source of variance. All jobs ran pinned to one socket and strictly one at a time, and a fixed single-thread calibration kernel ran before and after every job; a job started only when the kernel reached 380 queries per second (quiet value 500–600; 350 in a later, slower session) on two consecutive checks, and was re-run if it ended below that. Every configuration of the main campaign was measured in two passes and we report the faster one; the second pass and every re-run ran under this gate, whereas part of the first pass predates it and some of its jobs spanned both sockets (best-of-two mitigates this); recall is identical across passes for the deterministic methods and differs by a median of 0.003 for ACORN-$\gamma$ and 0.001 for Filtered-DiskANN, whose multi-threaded builds are not deterministic (pass spreads: Section IX); every reported point keeps the recall and QPS of the same run. Software: Faiss 1.15.1 (AVX2/AVX-512 dispatch), `diskannpy` 0.7.0, hnswlib 0.8.0, and ACORN at commit `c259f11` built with AVX2 and the patch of Section VIII.

## IV. A Cost Model for Filtered Search

The cost of answering a filtered $k$-NN query decomposes into three terms:

$$T = c_{\text{nav}} \cdot H \;+\; c_{\text{chk}} \cdot E \;+\; c_{\text{dist}} \cdot D, \tag{1}$$

where $H$ is the number of navigation steps (graph hops or centroid comparisons), $E$ the number of candidates whose admissibility is checked, and $D$ the number of distance computations to candidates, admissible or not (post-filtering computes them for inadmissible candidates too); $c_{\text{dist}}$ is one constant although a batched scan computes distances more cheaply than an index's scattered accesses. The three method families differ in which terms they can avoid. The exact scan over a label partition has $H{=}E{=}0$ and $D{=}|A_p|$. A predicate-agnostic method—IVF with a bitmap, ACORN, post-filtering—learns whether a candidate is admissible only by checking it, so $E > 0$. A label-indexed structure (a per-label partition, Curator, or the label-partitioned IVF of Section II) never meets an inadmissible candidate, so $E{=}0$ and $D$ is bounded by the admissible points near the query.

**Proposition 1 (crossover lower bound).** Suppose that admissibility is independent of position (the $\beta{=}0$ workloads) and that a predicate-agnostic method chooses the next candidate to check from positions and from the outcomes of earlier checks only, so that each newly checked candidate is admissible with probability $s$ independently of the past (an idealization: our quantile labels fix each label's size, under which the expected number of checks to the $k$-th admissible point is $k(n{+}1)/(m{+}1) \approx k/s$). To collect $k$ admissible candidates it checks, in expectation, at least $k/s$ candidates, so $T \geq c_{\text{chk}}\, k/s$, while the scan costs $c_{\text{dist}}\, s\, n$. The method can beat the scan only when $s\, n \geq \sqrt{(c_{\text{chk}}/c_{\text{dist}})\, k\, n}$, that is, a lower bound on the crossover admissible-set size grows as $\sqrt{n}$:

$$m^*(n) \;\geq\; \sqrt{\tfrac{c_{\text{chk}}}{c_{\text{dist}}}\, k\, n}. \tag{2}$$

*Proof sketch.* Each checked candidate is admissible independently with probability $s$; the number of checks until the $k$-th admissible one is negative binomial with mean $k/s$ (Wald's identity gives the same bound for any stopping rule that does not look ahead). Equating the two costs gives the threshold. □

The bound is loose—it ignores navigation and the extra candidates needed for recall—and it does not predict $m^*$ or that the measured crossover grows over any given range of $n$; it says only that such a method cannot keep a fixed crossover as $n$ grows. For pre- versus post-filtering under a fixed budget $B$, Mandarapu and Kunkunuru show that both are near-exact over a common selectivity band only when $B \geq \sqrt{kn}$, and observe the crossover selectivity falling as $1/\sqrt{n}$ on synthetic data, with SIFT1M matching [21]; Proposition 1 states it with measured cost constants for every method in its information model, which covers methods that learn admissibility only by checking candidates in a predicate-independent order (IVF with a bitmap, post-filtering, ACORN's checks). Plans that enumerate admissible ids—label partitions, posting lists, Curator's subtrees—are outside it. Section V measures the crossover across $n$ (H1) and whether $E$ and $D$ explain measured time (H2).

**Reachability of the admissible subgraph.** For an in-filter graph the relevant structure is the subgraph of live admissible nodes that search can traverse, given by each node's *effective admissible out-neighbors* as the search collects them. When deletions are only excluded through the predicate, deleted nodes keep their slots in these lists, and deletions concentrated on one region remove most of the live neighbors of the admissible nodes that remain there. Our first hypothesis was site percolation, under which the largest component vanishes once the mean effective degree falls to about one (Mandarapu and Kunkunuru measure a static percolation threshold $s_c \approx 0.83/M$, independent of $n$ [21]). Section VI measures effective degree and components per label and round (H3) and finds recall falling well before fragmentation, so the account we test causally, with local repairs, is the reach of search within its budget and the routes into a label's region.

**Proposition 2 (rebuild period).** Consider a label-specialized index rebuilt every $P$ rounds at cost $B > 0$, with points inserted since the last rebuild answered by an exact scan of a per-label delta. If a query at index age $a$ costs $t_0 + g\, a$—$g$ is the growth of the delta scan per round—then the cost per round with $Q$ queries per round is $B/P{+}Q\,(t_0{+}g\,(P{-}1)/2)$; for $Q, g > 0$ its continuous minimizer is

$$P^* = \sqrt{2B/(Q\, g)}, \tag{3}$$

the classical economic order quantity applied to rebuilds; what the measurements test is whether index aging is linear enough for it to choose the period. Among integer periods, rebuilding every round is optimal only when $Q \geq B/g$ (from $C(1) \leq C(2)$), while the continuous optimum over $P \geq 1$, $\max(1, P^*)$, reaches 1 only at $Q \geq 2B/g$. *Proof.* Differentiate in $P$, and compare $C(1)$ with $C(2)$. □ Section VII measures $t(a)$ and compares the best period with $P^*$ (H6).

## V. Static Filtered Search

### A. The exact scan is the baseline to beat (*Q1*)

An exact scan over the admissible points costs time proportional to $s \cdot n$ and nothing else; every index must beat it to be worth its build time, memory, and maintenance.

*The crossover grows with $n$ for predicate-agnostic indexes.* Proposition 1 bounds $m^*$ from below by a quantity that grows as $\sqrt{n}$; whether $m^*$ itself grows over a given range of $n$ is an empirical question. Figure 1 measures $m^*$ with labels independent of geometry ($\beta{=}0$) for $n$ from $3.16 \times 10^5$ to $10^7$. At recall 0.95, $m^*$ grows as $n^{0.49}$ for IVF (95% bootstrap interval over queries 0.47–0.52; 7 label and IVF-training seeds, one run each, give 0.41–0.48, a seed-level 95% interval of 0.42–0.46; the other exponents are point estimates, because the batched methods have no per-query times to resample), $n^{0.64}$ for ACORN, and $n^{0.41}$ for post-filtering; at $n{=}10^7$ IVF needs 54,344 admissible points to match the scan, ACORN 180,588, and post-filtering 128,836. The label-indexed structures behave the other way: Curator's

$m^*$ does not grow with $n$ ($n^{-0.05}$), and the label-partitioned IVF, which never touches an inadmissible entry, beats the scan at every measured size from $10^3$ admissible points up, at every $n$; at recall 0.99 its $m^*$ is about $10^3$ at $n{=}10^7$ (1,004). The bound is loose (with $c_{\text{chk}}/c_{\text{dist}} \approx 0.052$, $m^* \geq 2,277$ at $n{=}10^7$), but on DEEP every tested predicate-agnostic index has a positive fitted exponent and Curator a non-positive one, as its separation suggests.

*The cost terms account for the time.* Across 236 IVF configurations, query time is 0.073 ms plus 5.58 ns per entry checked against the bitmap and 108 ns per admissible distance ($R^2{=}0.967$, median relative error 19%, 90th percentile 61%). For ACORN, distance computations alone explain little of the time ($R^2{=}0.426$); in the counting build it performs a median of 209 predicate checks per distance computation, and checks and distances together explain its time with $R^2{=}0.980$. With independent labels, the number of entries IVF checks per collected neighbor at its cheapest setting with recall $\geq 0.95$ grows as $(1/s)^{0.517}$: above the bound's $1/s$ only through the extra lists needed for recall at large $s$, and approaching it as $s$ falls.

*Extrapolation.* A power law $T = c\, n^a s^b$ fitted on $n \leq 3.16\times 10^6$ predicts throughput at $n{=}10^7$ with median errors from 11% (label-partitioned IVF) to 25% (IVF); IVF's largest are at the lowest selectivities, where the check term $k/s$ takes over, a change of regime that the cost decomposition expresses.

*A 4096-dimensional check.* On arXiv ($\beta{=}0$, $n{=}10^5..10^6$) IVF's crossover again grows as $n^{0.52}$ (95% interval 0.51–0.53), and the label-partitioned IVF beats the scan at every measured size from $10^3$ admissible points up, at every $n$. A distance costs more here relative to a bitmap check ($c_{\text{chk}}/c_{\text{dist}}{=}0.016$ against 0.052 on DEEP), and, as Proposition 1 implies and as we predicted before the runs, the measured crossover is lower: IVF matches the scan at 3,859 admissible points on arXiv-1M, against 16,375 on DEEP-1M. Two results do not carry over: against the batched scan post-filtering's crossover does not grow with $n$ on these three sizes ($n^{-0.34}$; against a one-query-per-call scan it grows as $n^{0.43}$, so the exception reflects the batched scan's advantage at $d{=}4096$), and the cost decomposition fits IVF's time less well (median relative error 34%). On the dataset's real categories (arXiv-1M, median selectivity 0.149), only ACORN reaches recall 0.95, at 11.2× the faster exact scan; IVF, Curator, and post-filtering stop at 0.949, 0.926, and 0.842.

*Choosing the threshold.* We fix a scan-or-IVF threshold at the crossover measured on DEEP-1M and apply it unchanged, scoring each choice against the faster of the two at recall 0.95. Scaled with $\sqrt{n}$ it never picks the slower method on the other 17 DEEP workloads, while a fixed size or selectivity does on 2, and Milvus's 7% on 5; on the 138 workloads of 6 label and training seeds the calibration never saw, it picks wrong on 4 (at most 21% more cost), a fixed size on 8, a fixed selectivity on 16, and the 7% rule on 32 (up to 788%). On arXiv's 14 workloads it picks wrong on 5 (mean regret 31%); corrected by the square root of the change in $c_{\text{chk}}/c_{\text{dist}}$ from a dataset-free microbenchmark (random vectors and bitmaps), it picks wrong on 4 with mean regret 5% and at most 23%. We added this correction after seeing the uncorrected result, so it is an exploratory check, not a blind test. On a second machine (AMD EPYC 7513; $c_{\text{chk}}/c_{\text{dist}}{=}0.103$), a pre-registered test of the same correction picks right on all 23 workloads (uncorrected: 2 wrong; 7%: 4; Table V).

*Correlated labels.* With $\beta{=}0.9$ the same pattern appears when $s \cdot n$ is held fixed and $n$ grows from $10^6$ to $10^7$. At $10^4$ admissible points the scan wins at every $n$, and by more as $n$ grows: IVF, the best of the indexes compared there, reaches 0.5× the scan's throughput at recall 0.95 for $n{=}10^6$ and 0.1× for $n{=}10^7$, and every graph method fails to reach 0.95 at $n \geq 3.16 \times 10^6$; Curator, which indexes each label in its own subtree, keeps its ratio (0.4× the scan's throughput at both sizes). At $10^5$ IVF's lead shrinks with $n$: 3.6× the scan's throughput at $n{=}10^6$, and at $n{=}10^7$ 1.0× (re-measured in four paired blocks, 1.14×, 95% interval 1.04–1.25), although with independent labels it still wins there (2.1×). At $10^6$ admissible points indexes win: on DEEP-10M at $s{=}0.1$ IVF is 4.0× and Filtered-DiskANN 4.0× faster than the scan at recall 0.99, while Curator, designed for selective filters, does not reach 0.95 (maximum recall 0.895), as its authors' advice to pair it with a graph for common labels anticipates [23]. ACORN with $\gamma{=}100 \geq 1/s$ raises its maximum recall on SIFT1M ($s{=}0.01$, $\beta{=}0.9$) only from 0.946 to 0.956; in four rebuilds each, they reached 0.95 1 and 3 times.

When the predicate cannot be pre-partitioned, the exact scan either visits all $n$ rows and tests a bitmap or gathers the admissible rows from an id list (slower at $d{=}4096$, below); the *bitmap scan* answers 7.0 QPS on DEEP-10M versus 31 for the per-label partition, so with this bitmap scan arbitrary predicates move the crossover toward smaller admissible sets.

### B. Correlation reorders the methods (*Q2*)

Figure 2 varies the label–geometry correlation $\beta$ on SIFT1M at $s{=}0.1$. Post-filtering is among the fastest methods when labels are independent of geometry—854 QPS at 0.99 for $\beta{=}0$ on SIFT1M, within a few percent of IVF (859), and the fastest of all on DEEP-10M (506)—and the worst when they are not: its maximum recall falls to 0.903 at $\beta{=}0.9$ and 0.897 at $\beta{=}1$, because a query's unfiltered neighborhood then contains few admissible points. Filtered-DiskANN moves the other way—from 290 to 913 QPS at 0.99—because its per-label start points and label-preserving edges sit in the region where a correlated label lives. ACORN and IVF lose throughput as correlation grows (ACORN 721→243, IVF 859→489): for IVF the admissible points of a query's label live in lists far from the query, so more lists must be probed. Curator is insensitive to correlation (maximum recall 0.996 to 0.983), as a per-label index should be. On both SIFT1M and DEEP-10M, a benchmark that fixes $\beta{=}0$—uniformly random attributes, the common choice [12]—would thus favor post-filtering and penalize the label-specialized graph.

That penalty also depends on where queries lie. In every workload above a query asks for a uniformly random label, so with correlated labels it usually lies far from its label's region.

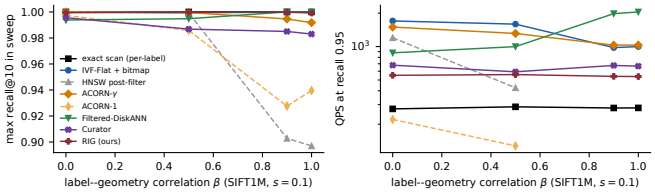



Fig. 2. Effect of label–geometry correlation on SIFT1M ($s$=0.1): maximum recall reached in each method's sweep (left) and QPS at recall 0.95 (right).

When a query instead asks, with probability $\rho$, for the label of its nearest neighbor, the ranking at $\beta$=0.9 changes: post-filtering, which does not reach recall 0.95 at $\rho$=0, answers 2,519 QPS at $\rho$=1 and becomes the second-fastest method after the label-partitioned IVF; IVF gains 1.77× and ACORN 1.87×, while Curator falls to 0.601× of its throughput and Filtered-DiskANN barely changes (1.06×). With independent labels $\rho$ matters little (IVF 1.01×, post-filtering 1.28×). A filtered benchmark therefore has to fix two correlations—of attributes with geometry and of queries with their filters—and its conclusions about post-filtering and graph indexes hold only for the second it chose.

### C. Real predicates need a dispatcher (*Q3*)

YFCC-10M's queries span four orders of magnitude of selectivity, and none of the indexes we evaluate on them reaches recall 0.99 by itself within its swept parameters. Table III breaks recall down by admissible-set size. The graph methods fail on small admissible sets: ACORN's recall is 0.027 below $10^3$ admissible points and 0.190 between $10^3$ and $10^4$, and reaches 0.995 only above $10^5$; RIG, whose entry points are a 1% sample, finds no admissible entry for the most selective queries (0.460); Filtered-DiskANN, which can follow only one tag of an AND and whose rare-tag subgraphs are poorly connected (Section VIII), stays below 0.632 except on the most common tags. IVF degrades least (0.956 below $10^3$). Meanwhile the per-query scan answers the smallest bucket at 5,608 QPS.

A common remedy is a selectivity-based dispatcher (ACORN's authors pre-filter below a selectivity of $1/\gamma$ [1]). In its oracle version, queries whose admissible set is smaller than a threshold $T$ go to the exact scan and the rest to the index, with $T$ and the index's parameter tuned jointly (Table III, right). Every method then reaches 0.99; IVF is fastest (311 QPS at $T$=32,768), ahead of ACORN (231), and the optimal thresholds of IVF, ACORN-$\gamma$, post-filtering, and RIG (32,768–65,536) are of the order of the crossover at $n$=$10^7$. Applied unchanged to 500 disjoint public queries (a later, slower session), the tuned choices reach recall 0.984–0.994 and keep the methods' order. Routing on cardinalities estimated under tag independence (a median underestimate of about 12× for AND queries) misroutes 0 of the 500 queries for IVF, since every AND query (at most 29,090 points) is already below the threshold. Curator is exact on selective queries (recall 1.000 below $10^3$ admissible points) and weaker on common

TABLE III

YFCC-10M (REAL TAGS). LEFT: RECALL BY ADMISSIBLE-SET SIZE $|A|$ AT EACH METHOD'S HIGHEST-RECALL SETTING (QUERIES PER BUCKET IN PARENTHESES). RIGHT: QPS AT THE RECALL TARGET 0.99 WITH A DISPATCHER THAT SCANS QUERIES WITH $|A| < T$, ORACLE (TUNED ON THESE QUERIES) AND HELD-OUT (THE SAME CHOICE ON 500 DISJOINT QUERIES, A LATER SESSION; ACHIEVED RECALL IN PARENTHESES, WHICH CAN FALL BELOW THE TARGET), AND THE HELD-OUT P99 LATENCY IN MS.

| | recall by $\|A\|$ | | | | | dispatched, target recall 0.99 | | |
|---|---|---|---|---|---|---|---|---|
| | $<10^3$ (123) | $10^3$–$10^4$ (112) | $10^4$–$10^5$ (113) | $10^5$–$10^6$ (116) | $\geq 10^6$ (36) | oracle QPS | held-out queries QPS (rec.) | p99 ms |
| IVF | 0.956 | 0.998 | 0.999 | 1.000 | 1.000 | 311 | 228 (0.994) | 14.2 |
| ACORN-$\gamma$ | 0.027 | 0.190 | 0.777 | 0.995 | 1.000 | 231 | 176 (0.984) | 30.4 |
| ACORN-1 | 0.015 | 0.189 | 0.748 | 0.980 | 0.989 | 82 | 62 (0.989) | 121.1 |
| post-filter | 0.435 | 0.721 | 0.932 | 0.997 | 0.997 | 200 | 144 (0.993) | 30.4 |
| F-DiskANN | 0.093 | 0.179 | 0.122 | 0.632 | 0.975 | 37 | 33 (0.989) | 261.3 |
| Curator | 1.000 | 1.000 | 0.998 | 0.965 | 0.953 | 38 | 34 (0.988) | 261.3 |
| RIG | 0.460 | 0.895 | 0.991 | 0.991 | 0.950 | 141 | 113 (0.992) | 43.3 |
| scan (per query) | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 11 | — | — |

ones (0.965 above $10^5$), so pairing it with another index for common queries—its authors recommend a graph [23]; we also pair it with IVF—is the fastest configuration we found (368 QPS with IVF, 346 with ACORN); on the held-out queries the pairings stay faster than scan plus IVF but miss the target slightly (recall 0.989 and 0.985, against 0.994), so their lead there is not at equal recall.

### D. Build cost and memory

The methods differ by up to 41× in build time and 10× in memory on DEEP-10M (as each runner measures it: index bytes, files, or Curator's RSS growth). IVF builds it in 60 s at the size of the data (3.9 GB); ACORN needs 2,109 s (8.9 GB) and Filtered-DiskANN 1,331 s (10.3 GB) on the same 32 threads, and Filtered-DiskANN's build is per label schema, so a new attribute means a new index. ACORN-1 builds 22–23× faster than ACORN-$\gamma$ but reaches a maximum recall of only 0.928 on SIFT1M and 0.967 on DEEP-10M at $s$=0.1, $\beta$=0.9 (ACORN-$\gamma$: 0.995 and 0.985). All build times and sizes are in the artifacts.

### E. Inside a database system

Inside a DBMS the optimizer, not the user, picks the access path. We issued the SIFT1M and DEEP-1M $n$-sweep queries as SQL to PostgreSQL 16 with pgvector 0.8.1, one backend per query. By default the optimizer chooses the HNSW index for 9 of 10 workloads without iterative scans, and the query then returns a recall between 0.012 and 0.939: the index yields the vector's nearest neighbors, the filter discards most of them, and the result is silently short. Only at $|A_p|$=$10^3$ does it pick the exact plan (B-tree, then sort). Forced into correct plans, the crossover reappears: HNSW with iterative scans overtakes the exact plan at about 13,275 admissible points on DEEP-1M (IVFFlat at 12,625), near IVF's crossover in Faiss at the same $n$ (16,375). In absolute terms the DBMS is about 36× (exact plan) and 15× (IVFFlat) slower than Faiss on DEEP-1M at $s$=0.1 (across sessions). Amanbayev et al. observed that pgvector's optimizer picks approximate index scans where an exact scan gives perfect recall at comparable latency [20]; we quantify the recall this choice loses. Milvus 2.6 [31]

(standalone, same workloads and parameters) has a dispatcher built in, and it is a fixed rule [19]: its HNSW search falls back to brute force whenever at least 93% of a segment's rows are filtered out, i.e. below a per-segment selectivity of 7% at any $n$ (other triggers: artifact). On DEEP-1M this brute force, a bitmap pass over the segment, answers 70.9–97.7 QPS at $s \leq 3.1\%$, while IVF_FLAT in the same server reaches recall $0.95$ at 163–186 QPS there, 1.67–2.49× faster down to $10^3$ admissible points, so at this $n$ the threshold is too high. Its default search parameters give recall 0.860–0.906 on the graph path.

### *F. Multi-threaded throughput*

We repeated three workloads with 28 search threads on one socket's physical cores, in each method's better mode (its native batched call or 28 threads of single-query calls), the better of two passes, against single-thread runs from the same session. ACORN scales 16.1–17.4×, post-filtering 16.7×, IVF 10.9–13.7×, RIG 5.9–7.2×, and the scan 5.4–10.1×, while a thread pool of Filtered-DiskANN's per-query calls is slower than one thread; post-filtering becomes the fastest method on DEEP-10M with $\beta$=0 (14,760 QPS at recall $0.95$) and ACORN overtakes IVF on SIFT1M (15,489 versus 11,399). The crossover rises with threads: at $n$=$10^7$ with independent labels, IVF's $m^*$ at recall $0.95$ grows from 54,344 to 143,307 and ACORN's from 180,588 to 234,677, because near the crossover the scan's speed-up exceeds IVF's by 2.9–3.6× (a ratio of two speed-ups, which cancels the session), while post-filtering's $m^*$ falls and the label-partitioned IVF still wins at $10^3$. Which method is fastest therefore depends on whether queries are served one at a time or batched across cores; the stream comparisons that follow are single-threaded.

## VI. FILTERED SEARCH UNDER UPDATES (**Q4**)

Table IV summarizes the last round of the three synthetic streams (per-round curves are in the artifacts). We group the methods by the update paths of Table II.

**Exact deletion (scan, IVF, RIG).** The per-label scan and IVF delete by removing entries, so nothing accumulates. IVF holds a maximum recall of 1.000 on SIFT-500K after $200\%$ turnover and 1.000 on DEEP-5M after $100\%$, at nearly unchanged throughput (2,444→2,321 and 557→546 QPS at recall $0.95$), for a total maintenance of 1.2 s and 12.4 s—cheaper than every other index of that session (Curator needs 5–9× as much, and the graph indexes two to three orders of magnitude more). RIG also deletes exactly (maximum recall ≥0.998 throughout) at 38–39× IVF's maintenance.

Curator deletes by revoking a point's labels, which removes it from the per-label subtrees although its vector stays in the shared tree; its quality also stays flat (maximum recall 0.987 on SIFT-500K and 0.990 under correlated deletions, 0.916 on DEEP-5M, where it never reaches $0.95$), at 5–9× IVF's maintenance.

**Deletion through the predicate (ACORN, post-filtering).** ACORN has no deletion routine; deleted points stay in the graph and are excluded by the query bitmap. Its graph therefore grows (3.0× resident nodes on SIFT-500K) and every deletion thins the live admissible points that in-filter navigation traverses. Under uniform deletions its maximum recall drifts from 1.000 to 0.993 and its throughput at recall $0.95$ from 1,855 to 656; on DEEP-5M it reaches $0.99$ at round 0 but not at the later measured rounds (5 and 10), and its throughput at $0.95$ falls from 427 to 197, while its incremental insertions cost 119× IVF's maintenance. *Correlated* deletions are far worse: when the stream deletes the geometric region of the highest labels (SIFT-500K-adv), ACORN's maximum recall collapses from 1.000 to 0.970 after four rounds and 0.575 after twenty. A control separates the update path from the data: rebuilt from scratch on exactly the live points, labels, and queries of round 20, ACORN reaches a maximum recall of 0.995 (1,300 QPS at recall $0.95$) on the correlated-deletion stream and 1.000 on the uniform one, against 0.575 and 0.993 for the incrementally maintained graph. The collapse and the drift therefore come from ACORN's update history, not from the live set; which part of that history matters (a rebuild also changes insertion order) is left to the instrumented runs below. Post-filtering with slot reuse keeps its recall under uniform churn but is slow at high recall (273 QPS at $0.95$ on SIFT-500K) and capped by the correlated predicate on DEEP-5M (maximum recall 0.913); under correlated deletions it loses ground (0.990→0.964).

**What collapses.** Section IV predicts that in-filter search fails when the subgraph of live admissible nodes thins out. The instrumented streams record, per label and round, the mean number of live admissible out-neighbors ACORN's level-0 search can collect from a node (its *effective degree*) and the share of the label's live nodes in the largest connected component of that subgraph. A label's maximum recall tracks its effective degree (0.997 at 16 or more, 0.643 between 4 and 8, 0.061 below 2). The collapse sets in near a degree of five rather than one, and before the subgraph fragments: labels with recall below $0.5$ still keep a mean share of 0.704 of their live nodes in one weakly connected component. Directed reachability accounts for it in our data: replaying ACORN's filtered descent exactly and searching the admissible subgraph from the entry it reaches, the share of a label's live nodes reachable tracks the label's recall (Figure 3; Spearman 0.93; on average 0.57 reachable and recall 0.57 with tombstones). A local repair that rewires each live node's level-0 list around its deleted neighbors (Section III) tests the mechanism: on SIFT-500K-adv it keeps maximum recall at 0.991 through round 12, against 0.670 without it, for 1.20–1.26× the maintenance of tombstones, but then falls again (0.878–0.982 at round 20 over 5 runs, against 0.546–0.615 with tombstones alone), and the labels that fail have dense, connected level-0 subgraphs (mean effective degree 15.8) and still about 156 live nodes above level 0. ACORN filters its upper levels too, and this repair leaves their lists pointing at deleted nodes, which can cut the route from the entry point into the label's region. Repairing every level supports this account (reachable share 0.99, against 0.63 when only the upper levels are repaired): maximum recall then stays at 0.995 at round 12 and 0.997–0.998 at round 20

in each of 5 runs—as high as a fresh build on the same live data; on three stream seeds measured at every round it ends at 0.99–1.00 but falls below 0.95 at 3 of 63 rounds (to 0.88), each time with the reachable share (0.89–0.94)—for 1.10–1.15× the maintenance of tombstones alone in the same session. In these streams ACORN's collapse is thus a property of its update path that a local consolidation removes, not of in-filter search. Measured again next to IVF in a later session (Table IV), the repair is no general cure: it leaves recall unchanged under uniform deletions on SIFT-500K and lowers it on DEEP-5M (0.943 against 0.983 with tombstones in the same run; over 6 runs of 5 stream seeds, the drop is $-0.044$ to $-0.020$), although a fresh build on the same live set reaches 0.994.

**What the repair changes.** Two variants test the repair policy. Repairing only the upper levels leaves the correlated-deletion collapse in place (0.623–0.628 on SIFT-500K-adv), so the level-0 lists must be repaired as well as the routes into them. Our repair re-ranks all candidates of a node that lost a neighbor and keeps the nearest; under uniform deletions, which touch nearly every node, it keeps 93% of a node's live neighbors but 83% of its long ones. That shortening does not explain the harm label by label (Spearman 0.03; the pre-registered test fails), but reachability does, in an exploratory analysis: re-ranking leaves 0.91 of a label reachable (slot-filling: 0.99), and the recall loss tracks the loss of the true neighbors' reachability (Spearman 0.77). Keeping a node's live neighbors in place and refilling only the deleted slots—the one policy changed—removes the harm on DEEP-5M (0.987, 278–299 QPS at recall 0.95, where re-ranking never reaches it) and reaches 1,301–1,372 QPS in SIFT-500K's last round (re-ranking, in an earlier session: 637), while still recovering most of the correlated collapse (0.983–0.988 against 0.998 with re-ranking; 0.85–0.96 on the three seeds). Where it restores recall, the re-ranking repair needs 255× IVF's maintenance for 621 against IVF's 1,795 QPS in the same run.

**Synchronous repair (EMA).** EMA repairs edges and Markers inside every deletion, and it holds maximum recall at $\geq$0.999 on each of the four main streams, including the correlated one where tombstoned ACORN collapses. The failure moves to throughput instead: on SIFT-500K-adv EMA answers 1,455 QPS at recall 0.95 before the deletions and 272 after 20 rounds (5.3× fewer, median over passes; IVF in the same session: 1.3×), against 1.8× under uniform deletions, at 408–440× IVF's maintenance on the SIFT streams.

**A real temporal stream.** On arXiv in time order (4096 dimensions, real categories, a sliding window that deletes the oldest papers) the ranking reverses. Graph indexes answer 7.2–8.4× (EMA) and, in their own session, 5.7–6.4× (ACORN) IVF's QPS at recall 0.95 in the last round, which is consistent with a distance costing far more than a navigation step at $d$=4096 (the data, labels, query–filter relation, and deletion order differ from the synthetic streams as well), and none collapses: deleting by age does not empty a label's region (ACORN's maximum recall stays $\geq$0.998). IVF, whose centroids were trained on the oldest window, loses throughput as the topics drift (240→130 QPS, the label-partitioned IVF 639→188); retraining every five rounds recovers it (271 QPS) for 4.4× IVF's maintenance (ACORN: 169×).

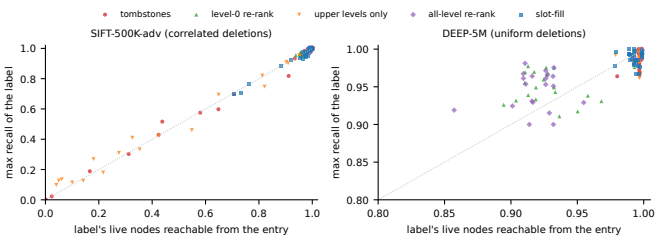



Fig. 3. ACORN under five update paths: a label's maximum recall against the share of its live nodes reachable from its queries' level-0 entry (ACORN's filtered descent replayed exactly), one point per label and measured round. Reading recall off reachability across paths is post hoc; the pre-registered tests are in Table V.

TABLE IV
LAST ROUND OF EACH STREAM: MAXIMUM RECALL / QPS AT RECALL 0.95 (1 THREAD; N/R: NOT REACHED) / TOTAL MAINTENANCE (S, 16 THREADS; FILTERED-DISKANN: ONE REBUILD PER ROUND). *LATER SESSIONS, WITH IVF RERUN ALONGSIDE (TWO PASSES, BEST PER CELL AS FOR THE OTHER ROWS).

| | SIFT-500K, round 20 | | | DEEP-5M, round 10 | | | SIFT-500K-adv, round 20 | | |
|---|---|---|---|---|---|---|---|---|---|
| | rec. | QPS | maint. | rec. | QPS | maint. | rec. | QPS | maint. |
| exact scan | 1.000 | 859 | 2.7 | 1.000 | 62 | 16.7 | 1.000 | 550 | 2.5 |
| label-part. IVF* | 0.999 | 6,922 | 0.8 | 0.999 | 2,544 | 5.8 | 1.000 | 5,261 | 0.9 |
| IVF | 1.000 | 2,321 | 1.2 | 1.000 | 546 | 12.4 | 0.999 | 1,866 | 1.2 |
| post-filter | 0.991 | 273 | 246.0 | 0.913 | n/r | 1,565.4 | 0.964 | 109 | 208.7 |
| ACORN-$\gamma$ | 0.993 | 656 | 276.1 | 0.982 | 197 | 1,481.1 | 0.575 | n/r | 275.1 |
| all-level repair* | 0.994 | 637 | 345 | 0.943 | n/r | 1,843 | 0.998 | 621 | 310 |
| EMA* | 1.000 | 848 | 535 | 1.000 | 101 | 2,753 | 1.000 | 272 | 508 |
| F-DiskANN | 0.999 | 2,490 | 1,043 | 0.996 | 828 | 8,197 | 1.000 | 3,771 | 1,084 |
| Curator | 0.987 | 885 | 9.7 | 0.916 | n/r | 57.5 | 0.990 | 1,498 | 11.1 |
| RIG | 1.000 | 724 | 46.1 | 0.999 | 202 | 486.5 | 0.999 | 578 | 46.1 |

**Rebuild (Filtered-DiskANN).** Freshly rebuilt, Filtered-DiskANN is the fastest method of the first session at recall 0.95 in the last round of all three synthetic streams (2,490 QPS on SIFT-500K, 828 on DEEP-5M, 3,771 on SIFT-500K-adv; at 0.99, 948 versus IVF's 1,188 on SIFT-500K), but each rebuild takes 52 s and 820 s on 16 threads, so staying fresh every round would cost (estimated as $R$ times the mean rebuild, within 1.0% of executed rebuilds on 3 streams) 1,043 s and 8,197 s—861× and 659× IVF's maintenance. Between rebuilds, inserted points are invisible to it.

## VII. TOTAL COST (Q5)

A deployment pays for maintenance and for queries. For a stream of $R$ rounds with $U$ updates and $Q$ filtered queries per round, method $X$ costs $M_X + Q\sum_r t_{X,r}$: its maintenance after the initial build plus its per-query time at the recall target in each round, at the effort tuned for that round and interpolated between measured rounds; Filtered-DiskANN's fresh-every-round maintenance is estimated. Figure 4 plots this cost against $Q/U$, relative to IVF in the same session, for every method that holds recall 0.95 at every measured round of every run; the others are named in each panel. On the three synthetic streams the label-partitioned IVF is the cheapest at every tested query rate (0.22–0.39× IVF). IVF is cheaper than ACORN, post-filtering, Curator, EMA, and RIG at every tested rate on

these streams; only a Filtered-DiskANN rebuilt every round undercuts it, from 3 queries per update on DEEP-5M and 10 on SIFT-500K-adv, where IVF slows down as deletions empty regions. The arXiv stream reverses the order with the query rate: the label-partitioned IVF is cheapest at 0.1 queries per update (0.59× IVF), and from one query per update graph indexes are cheapest, since their per-query advantage outweighs maintenance two orders of magnitude above IVF's; EMA is lowest (0.43× at 1, 0.19× at 100), also in each of 4 further blocks that ran every method in random order between two IVF anchors, at a cost 5–14% below repaired ACORN's (on Amazon reviews graphs take over from 3 queries per update); this needs the effort tuned per round, and with one effort for the whole stream ACORN and Filtered-DiskANN are cheapest there (Table V). The costs exclude the initial build, which a deployment pays once; including it changes the cheapest method in 0 of the 36 stream–rate cells. Timing queries end to end (stages per method: artifact) changes the cheapest method in none of the 36 cells (Table V). Memory is not costed, but it follows the update path: at a constant live set on SIFT-500K-adv (arXiv), the anonymous resident memory grows 1.8× (1.5×) for ACORN, which keeps deleted nodes, 2.6× (2.1×) for EMA, and 1.2× (1.2×) for IVF over the stream.

**Executed rebuild periods.** Between rebuilds, an exact per-label scan of the points inserted since the last build keeps them visible (Proposition 2). We executed this design on both SIFT streams for $P \in \{1, 2, 3, 4, 6, 10, 20\}$ rounds, measuring every round ($P$=20 never rebuilds within the stream; 2 runs per period). With the search effort chosen per round from the ground truth (an oracle), every period holds recall 0.95 on both streams, and the period that Proposition 2 computes from the *other* stream's rebuild time and delta growth costs at most 6.8% more than the best executed period in either run; re-estimating $B$ and $g$ from these runs instead changes the chosen period in 1 of 6 cases. A search effort fixed on the other stream does not transfer as well: $L$=64, enough on SIFT-500K-adv, misses 0.95 on SIFT-500K for 6 of the 7 periods in both runs, in up to 19 of 20 rounds. The period transfers between these two streams; the effort that holds a recall target must be checked on the stream itself.

TABLE V
PRE-REGISTERED FOLLOW-UP CHECKS (RULES FIXED BEFORE THE RUNS; ONE RUN PER CELL UNLESS STATED; XEON EXCEPT THE SECOND-MACHINE ROW)

| check | outcome |
|---|---|
| *Crossover and dispatch (Section V-A)* | |
| crossover, 7 seeds | IVF exponent 0.41–0.48, seed-level 95% interval 0.42–0.46 |
| threshold, 6 unseen seeds | wrong of 138: $\sqrt{n}$-scaled 4 (regret $\leq$21%), fixed size 8, fixed selectivity 16, 7% 32 ($\leq$788%) |
| second machine (EPYC 7513) | IVF exponent 0.39, crossover 1.6× the Xeon's; wrong of 23: own $\sqrt{n}$ rule 0, Xeon's corrected 0, uncorrected 2, 7% 4 |
| close verdicts, 4 paired blocks | of 10 index-vs-scan verdicts within 10%: 4 confirmed, 4 ties (interval includes 1), 2 reversed (both at $n$=$10^7$); feasibility at 0.95 holds for 7 of 10 edge cases (not for the three ACORN variants on SIFT1M) |
| half-query tuning; mean or worse pass | on the held-out half 0/71 index-vs-scan time orders change, but 5 chosen points miss 0.95 (by $\leq$0.011), so 3 recall-feasible verdicts change; 3 and 4 of 207 change with the mean or worse pass |
| *Update paths (Section VI)* | |
| repair, DEEP-5M, 6 runs | recall change $-0.044$ to $-0.020$ over 5 stream seeds, negative in every run |
| repair mechanism | entry account held (reach–recall $\rho$=0.93); long-edge account failed ($\rho$=0.03) |
| repairs, 3 SIFT-adv seeds, every round | at round 20 all-level 0.99–1.00, upper only 0.60–0.65, slot-fill 0.85–0.96, tombstones 0.58–0.64; reach–recall $\rho$ 0.92–0.98; the four ordering rules hold on 3/3, the every-round rule fails: all-level falls below 0.95 at 3/63 rounds (min 0.88) |
| multi-label stream | YFCC tags (AND); 0.95 held by the exact scan and by EMA, 14–27× slower than the scan (same round) with 3.2 h maintenance; max recall IVF 0.94, post-filtering 0.73, ACORN 0.48→0.38 |
| *Total cost (Section VII)* | |
| arXiv cost, 4 blocks | methods in random order between two IVF runs; from 1 query per update EMA is cheapest in every block (0.37–0.43× IVF; tombstoned ACORN 0.45–0.66×); IVF drift $\leq$23% |
| rebuild period, 2 runs | regret $\leq$6.8%; fixed $L$ misses 0.95 in 6/7 periods in both runs |
| end-to-end timing | cheapest method changes in 0 of 36 cells (EMA re-timed per query); relative cost moves $\leq$19% |
| every-round timing; one effort per stream | SIFT-adv, arXiv: interpolation changes no cheapest method; with one effort per stream, label-part. IVF stays cheapest on SIFT-adv; on arXiv ACORN (1 query per update) and rebuilt Filtered-DiskANN (3–100) undercut EMA; the round-0 effort later misses 0.95 for 4 of 6 methods on SIFT-adv |
| second temporal stream | Amazon reviews, 384-d: label-part. IVF cheapest up to 1 query per update (arXiv: 0.3), EMA cheapest from 3, as predicted (data, embedding model, and drift also differ) |
| *Service and re-execution* | |
| concurrent updates | 1 writer, 8 readers, every method behind a fair reader-writer lock; p99 over quiet (SIFT, SIFT-adv): IVF 9.0, 6.4×; label-part. IVF (lock per label) 1.2, 1.4×; post-filtering 32.6, 18.2× and EMA 87.6, 25.3×, mostly the wait for a write batch (neither library permits updates during searches); 0 stale or invalid ids |
| re-execution, 132 random jobs | recall identical for deterministic methods (111/111 files), within tolerance for 221/236 others; QPS ratio median 1.01 (p10–p90 0.94–1.12); 239/254 pairwise verdicts agree |

## VIII. LESSONS FOR EVALUATING FILTERED SEARCH (**Q6**)

A shared machine and eleven methods from seven code bases surfaced failure modes of the *evaluation* as much as of the methods.

**Machine contention.** Our calibration kernel varied between 6 and 634 queries per second over the 25 days of our measurements. In one early pass the scan and IVF arms of seven SIFT workloads ran at a quarter of the kernel's quiet value and, re-run, were up to 2.9× faster, enough to reverse which method wins at $s \cdot n = 10^5$; hence the protocol of Section III.

**Defects and limits of baseline implementations.** ACORN's incremental `add` writes one entry past a full upper-level neighbor list and corrupts the graph (we truncate to capacity, as Faiss HNSW does), and its documented build compiles scalar kernels (our AVX2 build searches 1.1–1.4×

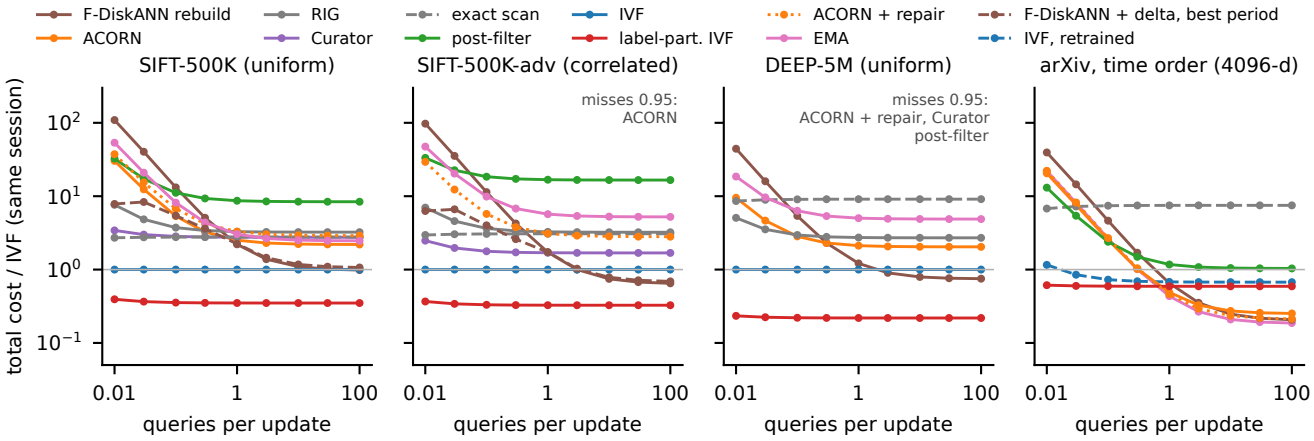



Fig. 4. Total cost (maintenance after the initial build plus queries at recall 0.95, per-query time interpolated between measured rounds) relative to IVF in the same session, against queries per update. A method appears only if it reaches 0.95 at every measured round of every run; the others are named. Medians over runs; efforts are tuned per measured round (the rebuild policies' too); the rebuilt Filtered-DiskANN's maintenance is estimated (Section VI).

faster at identical recall); `diskannpy` pads short filtered results with uninitialized identifiers; and Filtered-DiskANN with one label per point stays below recall 0.2 at $s$=0.01 (at most 0.168), as a user report [32] and recent benchmarks [23], [13] also find.

**Harness pitfalls.** Our own bugs changed conclusions before we caught them: a stale index sized its request by overall rather than per-label survival; Milvus indexes were built while segments still merged; and, found by an independent audit, a held-out check ignored recall and a concurrency test let hnswlib and EMA mutate during searches. The superseded runs are kept with their reasons. Duplicate vectors charge misses for live copies (our streams use distinct descriptors); every measured number here is a macro from the analysis output.

**Interpolation decides close comparisons.** QPS at a recall target is interpolated between the two bracketing measured points, which on doubling grids differ by a median factor of 1.69 in QPS for IVF (3.29 for post-filtering); in 30 of 177 index-versus-scan comparisons the verdict changes between them. Re-measured with knob steps of $\sqrt{2}$, 20 of the 23 covered verdicts stand and 3 reverse (2 within 5% of one); claims that rest on a factor below the bracket should be measured. Best-of-two raises QPS over the mean of two passes by a median 1.03× for the scan and 1.01–1.02× for the graphs, and rarely changes a verdict (Table V).

**Simple baselines.** The exact scan, IVF with a bitmap, a label-partitioned IVF where labels are known at build time, and a dispatcher each changed a conclusion here; evaluations should report them.

## IX. Limitations and Threats to Validity

**Measurement.** All results come from one shared machine, except the second-machine replication of Table V. The gating protocol of Section III removes runs taken under gross contention, but part of the first pass predates the gate, and differences below the pass-to-pass spread (90th percentile 1.10×) should not be over-read. The machine's single-thread speed also varied between sessions (by about a third in the last session); every ratio we report is therefore formed within one session, and throughputs from different sessions are compared only by their ranking. The cost frontier combines such ratios across sessions, assuming that session effects are alike across methods. With 500 queries, the standard error of mean recall is at most 0.009 at recall $\geq 0.95$.

**Tuning on the reported queries.** Sweeps and the dispatcher's threshold are evaluated on the reported queries, as is customary; applied unchanged to 500 disjoint YFCC queries, the dispatcher's choices reach a recall of 0.984–0.994 at the 0.99 target. On the held-out halves of Table V, time orders held but 5 of 71 chosen points missed the target. Reported operating points are therefore optimistic, not necessarily by the same amount for every method, and recall at the target is not guaranteed on new queries.

**Workloads.** Synthetic labels are quantiles of one random projection, one per point; real predicates come from YFCC-10M's tags and arXiv's categories; $k$=10 throughout. In the three synthetic streams insertions follow the base distribution, so IVF's centroids meet no drift (arXiv's do). Synthetic deletions are either uniform or concentrated along one direction; arXiv deletes the oldest papers.

**What is not timed.** Predicate evaluation is excluded: with our numpy path it costs 3.2 ms per YFCC query, about as much as the dispatched IVF search (99%), while an AND of Roaring bitmaps takes 0.026 ms (0.266 ms with the admissible ids), so it is a property of the attribute index and is reported separately; the end-to-end check of Table V times per-label bitmaps kept current by the updates.

**Implementations and coverage.** We evaluate public implementations through their native or Python interfaces (Filtered-DiskANN through `diskannpy`, which has no dynamic filtered index; ACORN with our patch, AVX2, $\gamma$=12). Beyond

the methods of Section X that have no update path, we do not evaluate EMA on static workloads or disk-resident indexes; Curator runs with its authors' YFCC parameters; updates and queries alternate by round except in one concurrency test (Table V).

**Explanations and scope of claims.** The cost model's terms are measured for IVF and ACORN (Section V-A); for the other methods the crossover's growth rests on fitted exponents over four sizes of one dataset (on arXiv IVF's exponent recurs), and Proposition 1 is a bound under independent labels, not a prediction of $m^*$. Table cells (as Table IV) report the better of two passes, the cost frontier each run whole. The bootstrap intervals resample queries only; across passes IVF's exponent moves by at most 0.02, across seeds it lies between 0.41 and 0.48. The study began as the evaluation of a new index, and its questions followed when simple baselines proved competitive. The follow-up campaign's hypotheses were stated with falsification criteria before it ran: operationalizing Proposition 1 as an exponent of at least $0.5$, ACORN's estimate (0.64) exceeds it, IVF's seed-level interval (0.42–0.46) lies below it, and post-filtering's (0.41) fails, so the data support the bound and the exponent's sign, not a $\sqrt{n}$ law (the measured $m^*$ lies well above the bound, so its local slope can be below $0.5$). The entry-point test, the all-level repair, the dense sweeps, and the PostgreSQL, Milvus, and arXiv experiments were added after first results and are exploratory; the predicted effects of cardinality-estimation error and best-of-two selection were not found. Later follow-ups (Table V) were pre-registered with their decision rules in the artifact before their runs; one of two mechanism accounts failed, and so did the every-round test of the all-level repair.

**Conflict of interest.** RIG is the author's own, unpublished design. It was developed on SIFT1M and DEEP with the query sets used here, which may favor it; we report its losses and draw no conclusion that depends on it.

## X. Related Work

**Experimental studies of filtered search.** The closest studies (Table I) evaluate static data and do not control attribute–vector correlation: Zhu et al. [11] (fifteen hybrid-query algorithms, up to $10^8$ points), Li et al. [12] (twelve methods, few of which support updates), Shi et al. [13] (ten label-filter methods with unified tuning, including pre-filter brute force and post-filtered IVF-PQ), and Iff et al. [15] (arXiv embeddings with eleven attributes); a tutorial surveys the area [35]. Inside a PostgreSQL-compatible engine, Lu et al. [16] find that system overheads and filter checks change which method wins; Amanbayev et al. [20] find IVF-Flat ahead of HNSW at low selectivity in Faiss, Milvus, and pgvector; Lim et al. [33] argue that selectivity and correlation are unstable proxies for hardness; Sanca and Ailamaki [34] characterize scan-versus-index access paths; the winning Big-ANN'23 filtered entry chose between a linear scan and a graph by tag cardinality [17]; Mandarapu and Kunkunuru model plan choice as a phase transition with a budget threshold at $\sqrt{kn}$ [21]; and FAVOR routes queries by estimated selectivity [36]. We do not claim the first scan or IVF baseline, the first controlled correlation, or the $1/\sqrt{n}$ scaling of the crossover selectivity [21]; we add the joint treatment of filters and updates, the crossover's growth across index families (IVF, ACORN, post-filtering) its reappearance inside pgvector, its absence for label-indexed structures, and the correlated-deletion failure and its repair. Studies of deletion in graph indexes [38], [39] do not vary how deletions relate to an attribute; our rebuild on the same live set shows that the correlated failure comes from the update path.

**Streaming ANN.** Without filters, the Big-ANN'23 streaming track runs insert/search/delete runbooks and CANDY [18] continuous ingestion; FreshDiskANN [8], SPFresh [9], in-place DiskANN updates [10], and incremental IVF [37] evaluate their own update paths; Xiao et al. [38] document unreachable points in HNSW under replaced updates; Yamashita et al. [39] formalize deletion and rebuilding in graph indexes; MERIT repairs graphs in place on deletion [40]; Mandarapu and Kunkunuru find signal-triggered repair no better than scheduled repair [41]; and Das et al. separate staleness from filtered navigability under concurrent updates [42]. Dynamic *filtered* indexes—DIGRA [43], Zhang et al. [44], DSG [45], WoW [46], Curator [23], EMA [25]—evaluate narrower updates (Table I), and VecBench [47] reports update phases as latencies. We know of no study that tracks the recall and throughput of several filtered methods, with exact-scan and IVF baselines, while insertions and deletions interleave, or that varies how deletions correlate with the attribute.

**Methods not evaluated.** Range-filter structures (SeRF [5], iRangeGraph [6], UNIFY [7], window filters [48]) target range predicates; label-centric methods include NHQ [4], UNG [3], and CAPS [49]; SIEVE [50] tunes a collection of indexes for arbitrary predicates; NaviX [51] and Compass [52] serve general predicates inside database systems. We evaluate only methods whose released code supports updates, and PostgreSQL and Milvus among database systems.

## XI. Conclusion

Four guidelines follow, each with the conditions under which it held. *(i)* Keep an exact pre-filter scan, and set the dispatch threshold for the collection size, threads, and hardware at hand: scaled with $\sqrt{n}$ and corrected by a dataset-free microbenchmark it transferred across DEEP sizes, seeds, and a second machine; a fixed selectivity did not. *(ii)* Measure both correlations—attributes with embeddings, and queries with their filters—before choosing between post-filtering and a filter-aware index. *(iii)* Choose the index for a stream by its total cost at the target recall over the expected ratio of queries to updates, not by query speed; on both temporal streams the cheapest method changed with that ratio, and on arXiv IVF needed retraining. *(iv)* Judge an update path by recall *and* throughput under both uniform and correlated deletions: a repair that removed one failure caused another. Open problems: deletion paths robust to both deletion patterns, and dispatchers that adapt to $n$, threads, and drift. Artifacts: https://github.com/suanlab/filtered-ann-updates-bench.

## ACKNOWLEDGMENT

**Use of AI-generated content.** In accordance with the IEEE policy on AI-generated content, we disclose that AI systems were used substantially in this work, through the Claude Code agent: Claude Opus 5 and Claude Fable 5 (Anthropic; August–September 2026) for the design of RIG, its kernels, and early experiments; Claude Opus 4.8 (September 2026) for revisions of that work; and Claude Opus 5.5 (October 2026) for everything else listed here. They implemented the harness, workload generators, baseline drivers and patches, and RIG (Sections II–III); ran and analyzed the experiments (Sections V–VII); generated every figure and table from the raw results; they identified the implementation defects reported in Section VIII; at the author's request they proposed the cost model of Section IV, the hypotheses of the follow-up campaign with their falsification criteria, and the experiments that test them, which the author approved before they were run; they drafted the text of all sections; and AI agents, including Codex (OpenAI), performed internal reviews and audits of the manuscript, code, and results. The human author defined the research questions, made the design and scope decisions, reviewed every result and revision, and takes full responsibility for the content. No AI system is listed as an author.

## REFERENCES

[1] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manag. Data*, vol. 2, no. 3, pp. 1–27, 2024, doi: 10.1145/3654923.

[2] S. Gollapudi et al., "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. (WWW)*, 2023, pp. 3406–3416, doi: 10.1145/3543507.3583552.

[3] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–27, 2024, doi: 10.1145/3698822.

[4] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 15738–15751, doi: 10.52202/075280-0692.

[5] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 1–26, 2024, doi: 10.1145/3639324.

[6] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–26, 2024, doi: 10.1145/3698814.

[7] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "UNIFY: Unified index for range filtered approximate nearest neighbors search," *Proc. VLDB Endow.*, vol. 18, no. 4, pp. 1118–1130, 2024, doi: 10.14778/3717755.3717770.

[8] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search," arXiv:2105.09613, 2021.

[9] Y. Xu et al., "SPFresh: Incremental in-place update for billion-scale vector search," in *Proc. 29th ACM Symp. Operating Syst. Princ. (SOSP)*, 2023, pp. 545–561, doi: 10.1145/3600006.3613166.

[10] H. Xu, M. D. Manohar, P. A. Bernstein, B. Chandramouli, R. Wen, and H. V. Simhadri, "In-place updates of a graph index for streaming approximate nearest neighbor search," arXiv:2502.13826, 2025.

[11] J. Zhu et al., "An experimental evaluation of hybrid querying on vectors," *Proc. VLDB Endow.*, vol. 19, no. 2, pp. 183–195, 2025, doi: 10.14778/3773749.3773757.

[12] M. Li, X. Yan, B. Lu, Y. Zhang, J. Cheng, and C. Ma, "Attribute filtering in approximate nearest neighbor search: An in-depth experimental study," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025, doi: 10.1145/3769763.

[13] J. Shi, Y. Cai, and W. Zheng, "Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study," arXiv:2509.07789, 2025.

[14] Y. Hou, J. Li, X. Fu, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation: Benchmarking LLMs as semantic encoders," arXiv:2403.03952, 2024.

[15] P. Iff, P. Brügger, M. Chrapek, D. Kochergin, M. Besta, and T. Hoefler, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," in *Proc. 49th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2026, pp. 610–622, doi: 10.1145/3805712.3809731.

[16] D. Lu et al., "An in-depth study of filter-agnostic vector search on a PostgreSQL database system," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 1–26, 2026, doi: 10.1145/3802011.

[17] H. V. Simhadri et al., "Results of the Big ANN: NeurIPS'23 competition," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, pp. 76394–76408, doi: 10.52202/085713-2303.

[18] X. Zeng, Z. Wu, X. Hu, X. Shi, S. Sun, and S. Zhang, "CANDY: A benchmark for continuous approximate nearest neighbor search with dynamic data ingestion," arXiv:2406.19651, 2024.

[19] Zilliz, "Knowhere v2.6.15, the vector search engine of Milvus 2.6.18," `src/index/hnsw/impl/IndexConditionalWrapper.h`, https://github.com/zilliztech/knowhere/tree/v2.6.15, accessed Oct. 2026.

[20] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," arXiv:2602.11443, 2026.

[21] M. Mandarapu and S. Kunkunuru, "Filtered ANN as a phase transition: When selectivity-estimation error causes plan regret," arXiv:2606.16341, 2026.

[22] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[23] Y. Jin et al., "Curator: Efficient vector search with low-selectivity filters," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–27, 2026, doi: 10.1145/3786635.

[24] M. Douze et al., "The Faiss library," *IEEE Trans. Big Data*, vol. 12, no. 2, pp. 346–361, 2026, doi: 10.1109/TBDATA.2025.3618474.

[25] M. Li, B. Lu, J. Cheng, and C. Ma, "EMA: Approximate nearest neighbor search with general attribute filtering and dynamic updates," arXiv:2606.00734, 2026.

[26] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, 2011, doi: 10.1109/TPAMI.2010.57.

[27] A. Babenko and V. Lempitsky, "Efficient indexing of billion-scale datasets of deep descriptors," in *Proc. CVPR*, 2016, pp. 2055–2063, doi: 10.1109/CVPR.2016.226.

[28] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," *Inf. Syst.*, vol. 87, p. 101374, 2020, doi: 10.1016/j.is.2019.02.006.

[29] B. Thomee, D. A. Shamma, G. Friedland, B. Elizalde, K. Ni, D. Poland, D. Borth, and L.-J. Li, "YFCC100M: The new data in multimedia research," *Commun. ACM*, vol. 59, no. 2, pp. 64–73, 2016, doi: 10.1145/2812802.

[30] H. Jégou, R. Tavenard, M. Douze, and L. Amsaleg, "Searching in one billion vectors: Re-rank with source coding," in *Proc. ICASSP*, 2011, pp. 861–864, doi: 10.1109/ICASSP.2011.5946540.

[31] J. Wang et al., "Milvus: A purpose-built vector data management system," in *Proc. Int. Conf. Manage. Data (SIGMOD)*, 2021, pp. 2614–2627, doi: 10.1145/3448016.3457550.

[32] Microsoft DiskANN GitHub issue #550, "[Question] Low Recall in Filtered-Disk ANN on Sift Dataset-1M with Uniform and Zipf Distributions," https://github.com/microsoft/DiskANN/issues/550, accessed 2026.

[33] M. Lim, D. Kim, M. Kim, and J. Do, "Revisiting filtered ANN benchmarks: A hardness-controlled benchmark generator for realistic evaluation," *Proc. VLDB Endow.*, vol. 19, no. 10, pp. 2763–2776, 2026, doi: 10.14778/3828612.3828630.

[34] V. Sanca and A. Ailamaki, "Efficient data access paths for mixed vector-relational search," in *Proc. 20th Int. Workshop Data Manage. New Hardware (DaMoN)*, 2024, pp. 1–9, doi: 10.1145/3662010.3663448.

[35] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportuni-

ties," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5488–5492, 2025, doi: 10.14778/3750601.3750700.
[36] J. Song et al., "FAVOR: Efficient filter-agnostic vector ANNS based on selectivity-aware exclusion distances," arXiv:2605.07770, 2026.
[37] J. Mohoney et al., "Incremental IVF index maintenance for streaming vector search," arXiv:2411.00970, 2024.
[38] W. Xiao, Y. Zhan, R. Xi, M. Hou, and J. Liao, "Enhancing HNSW index for real-time updates: Addressing unreachable points and performance degradation," arXiv:2407.07871, 2024.
[39] T. Yamashita, D. Amagata, and Y. Matsui, "How should we evaluate data deletion in graph-based ANN indexes?" presented at the *NeurIPS 2025 Workshop on Machine Learning for Systems*, 2025, arXiv:2512.06200.
[40] Z. Wu et al., "MERIT: Efficient in-place deletion for dynamic graph-based approximate nearest neighbor indexes," arXiv:2607.29173, 2026.
[41] M. Mandarapu and S. Kunkunuru, "When to repair a graph ANN index: A matched-budget negative result, and the interpolated-baseline trap that hid it," arXiv:2607.00728, 2026.
[42] N. Das, K. Devadkar, and A. Godbole, "Recall-bounded staleness for filtered multi-tenant ANN graph indexes under concurrent updates," arXiv:2610.15130, 2026.
[43] M. Jiang et al., "DIGRA: A dynamic graph indexing for approximate nearest neighbor search with range filter," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725399.
[44] F. Zhang et al., "Efficient dynamic indexing for range filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725401.
[45] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025, doi: 10.14778/3748191.3748193.
[46] Z. Wang, J. Zhang, and W. Hu, "WoW: A window-to-window incremental index for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–27, 2025, doi: 10.1145/3769843.
[47] X. Zhang, C. Zhang, J. Fan, G. Li, and X. Du, "VecBench: A controllable benchmark for filtered vector search," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 1–27, 2026, doi: 10.1145/3802125.
[48] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 12469–12490.
[49] G. Gupta, J. Yi, B. Coleman, C. Luo, V. Lakshman, and A. Shrivastava, "CAPS: A practical partition index for filtered similarity search," arXiv:2308.15014, 2023.
[50] Z. Li, S. Huang, W. Ding, Y. Park, and J. Chen, "SIEVE: Effective filtered vector search with collection of indexes," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4723–4736, 2025, doi: 10.14778/3749646.3749725.
[51] G. Sehgal and S. Salihoğlu, "NaviX: A native vector index design for graph DBMSs with robust predicate-agnostic search performance," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4438–4450, 2025, doi: 10.14778/3749646.3749704.
[52] C. Ye, X. Yan, and E. Lo, "Compass: General filtered search across vector and structured data," arXiv:2510.27141, 2025.